



**UNIVERSIDADE FEDERAL DO TOCANTINS –
UFT CAMPUS DE PALMAS
PROGRAMAÇÃO ORIENTADA A OBJETOS - POO**

João Pedro de Oliveira Leite Gonzaga

PROJETO DE XADREZ: RELATÓRIO

**Palmas – TO
2026**

1. Introdução

Este projeto consiste na implementação de um jogo de xadrez em Java utilizando os conceitos de Programação Orientada a Objetos (POO). O objetivo foi desenvolver um sistema capaz de representar uma partida completa de xadrez, aplicando as principais regras do jogo.

2. Decisões de Projeto

Utilização da Programação Orientada a Objetos

O sistema foi desenvolvido seguindo os princípios da Programação Orientada a Objetos, permitindo uma melhor organização do código e facilitando a manutenção e expansão do projeto. Foram utilizadas classes específicas para representar os principais elementos do jogo, como tabuleiro, peças, posições e controle da partida.

Hierarquia de Classes

Foi criada uma classe abstrata chamada *Piece*, responsável por representar uma peça genérica do tabuleiro. A partir dela foi criada a classe *ChessPiece*, que adiciona características específicas do xadrez, como cor e quantidade de movimentos realizados. As peças do jogo (*King*, *Queen*, *Rook*, *Bishop*, *Knight* e *Pawn*) herdam da classe *ChessPiece*, permitindo reutilização de código e aplicação do conceito de herança.

Separação de Responsabilidades

O projeto foi dividido em componentes com responsabilidades específicas:

Board: gerenciamento do tabuleiro.

Position: representação de posições internas.

ChessPosition: representação das posições no formato tradicional do xadrez.

ChessMatch: controle da partida e regras do jogo.

UI: interação com o usuário.

Classes das peças: definição dos movimentos de cada peça.

Essa divisão melhora a organização do sistema e reduz o acoplamento entre os componentes.

Tratamento de Exceções

Foram utilizadas exceções personalizadas (*BoardException* e *ChessException*) para tratar situações inválidas, como movimentações ilegais, posições inexistentes ou tentativas de realizar jogadas que violem as regras do jogo.

3. Regras Implementadas

Movimentação das Peças

Foram implementados os movimentos específicos de todas as peças do xadrez:

Rei, Dama, Torre, Bispo, Cavalo e Peão

Cada peça possui sua própria lógica de movimentação através do método *possibleMoves()*.

Captura de Peças

O sistema permite a captura de peças adversárias, removendo elas do tabuleiro e adicionando à lista de peças capturadas.

Controle de Turnos

A partida alterna automaticamente entre os jogadores das cores branca e preta após cada jogada válida.

Validação de Jogadas

Antes de realizar uma movimentação, o sistema verifica:

1. Se existe uma peça na posição de origem.
2. Se a peça pertence ao jogador atual.
3. Se o movimento é permitido para aquela peça.
4. Se a jogada não coloca o próprio rei em xeque.

Xeque

Foi implementada a detecção de xeque, identificando quando o rei de um jogador está sob ameaça de captura por uma peça adversária.

Xeque-Mate

Foi implementada a verificação de xeque-mate através da simulação de todos os movimentos possíveis do jogador em xeque. Caso nenhum movimento elimine a ameaça ao rei, a partida é encerrada.

Roque

Foram implementadas as duas formas de roque:

Roque pequeno e roque grande, o sistema verifica se o rei e a torre envolvidos nunca se moveram antes da execução do movimento.

Movimento Inicial Duplo do Peão

O peão pode avançar duas casas em seu primeiro movimento

En Passant

Foi implementada a captura especial "*En Passant*", respeitando as regras oficiais do xadrez.

Promoção do Peão

Quando um peão alcança a última linha do tabuleiro, o sistema permite que o jogador escolha a peça para promoção entre a dama, torre, bispo e cavalo.

4. Conclusão

O projeto permitiu aplicar conceitos fundamentais de Programação Orientada a Objetos, como herança, encapsulamento, abstração e polimorfismo. Além disso, foi possível implementar as principais regras oficiais do xadrez, resultando em um sistema funcional e organizado, capaz de simular partidas completas utilizando o console como interface para o usuário.